

07

비주얼 스튜디오 코드로 다루는 깃과 깃허브

07-1 저장소 만들기

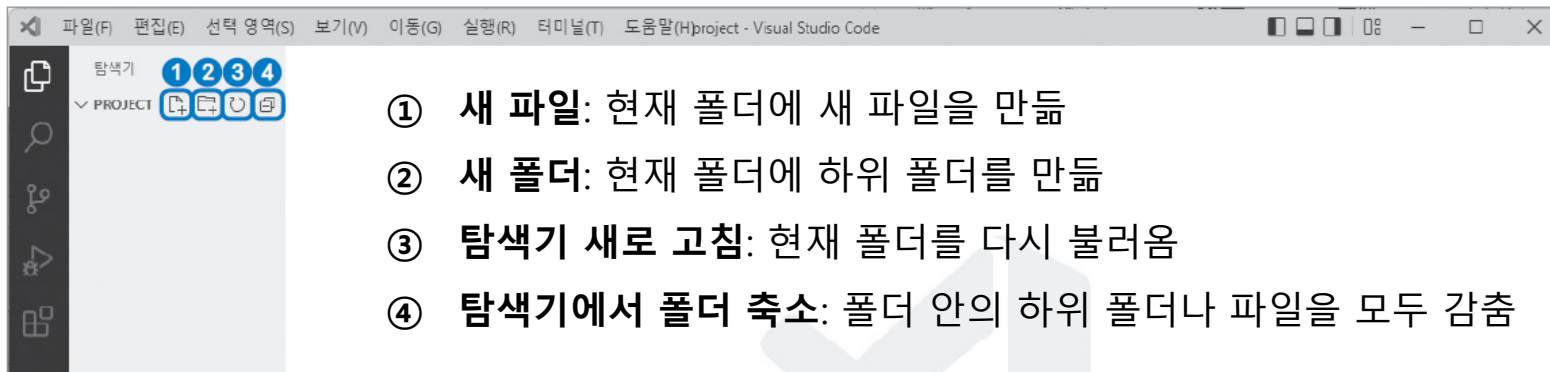
07-2 버전 만들기

07-3 깃허브에 연결하기

07-4 브랜치 관리하기

새 폴더 만들고 작업 폴더 추가하기

1. VS Code에서 버전 관리를 연습해 보기 위해 'project'라는 새로운 폴더를 만들
2. VS Code를 실행한 후 [파일] → [폴더 열기]를 선택
그리고 새로 만든 [project] 폴더를 선택하고 [폴더 선택]을 클릭
macOS를 사용한다면 VS Code가 열린 상태에서 [project] 폴더를 VS Code 화면 위로 드래그하면 됨
3. [project] 폴더가 작업 폴더로 추가되면 VS Code의 왼쪽 탐색기 창에 [PROJECT]라는 폴더 이름이 나타남
아직 아무 파일도 만들지 않아서 폴더 이름 아래쪽은 비어 있고, [PROJECT] 폴더 이름 위로 마우스 포인터를 올려놓으면 작은 아이콘들이 나타남



저장소 초기화하기

1. VS Code에서 버전 관리와 관련된 명령은 화면 왼쪽의 사이드바에서  (소스 제어)를 클릭하면 나타나는 소스 제어 창에서 이루어짐
사이드바에서  를 클릭하면 리포지토리 초기화부터 하라고 알려 주므로 [리포지토리 초기화]를 클릭
2. 소스 제어 창의 내용이 바뀌고 소스 제어 창의 오른쪽 위에 있는  (보기 및 기타 작업)을 클릭하면 앞에서 공부했던 풀이나 푸시, 커밋과 같은 명령도 볼 수 있음
VS Code의 맨 아래 상태 표시줄을 보면 가장 왼쪽에  main (main 브랜치)가 있음
지역 저장소가 만들어졌고 현재 main 브랜치에 있다는 뜻

깃 설정하기

1. [터미널] → [새 터미널]을 선택하거나 {{ Ctrl + Shift +  }}(윈도우) 또는 {{ Ctrl + Shift +  }}(맥)을 눌러서 터미널 창을 엽니다
VS Code에서 터미널 창을 사용할 수 있어서 편리
2. 터미널 창에 다음과 같이 입력
만일 저장소마다 다른 계정을 사용한다면 다음 소스에서 --global을 빼고 입력

```
git config --global user.name "사용자 이름"  
git config --global user.email "이메일 주소"
```

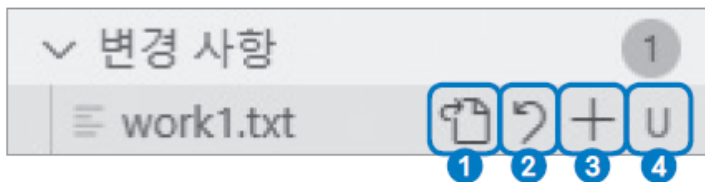
3. 사용자 설정이 끝나면 터미널 창 오른쪽 위에 있는 [X]를 눌러 터미널 창을 닫음

파일 만들기

1. 탐색기 창에서 [PROJECT] 폴더 이름 위로 마우스 포인터를 올린 후  (새 파일)을 클릭하면 새 파일이 만들어지면서 파일 이름을 입력할 수 있음
'work1.txt'라고 입력하고 {{ Enter }}를 누름
2. work1.txt 파일이 만들어지면서 오른쪽에 방금 만든 work1.txt 파일을 편집할 수 있는 창이 열림
실제로는 복잡하고 긴 소스를 사용하겠지만 여기에서는 간단히 '1'을 입력
{{ Ctrl + S }}(윈도우)나 {{ Control + S }}(맥)를 눌러 파일을 저장
3. 파일에 내용을 입력하는 순간  처럼 소스 제어 아이콘에 '1'이라는 숫자가 표시될 것
파일 1개에 수정 사항이 있다는 뜻

소스 제어 창 살펴보기

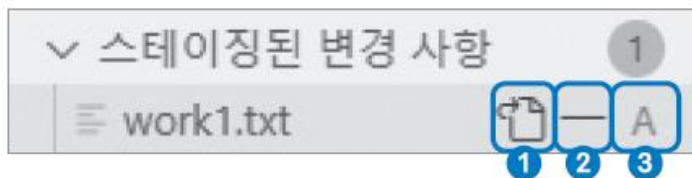
1. 앞에서 새로운 파일을 만들었기 때문에 소스 제어 아이콘에는 '1'이라고 표시되어 있음
소스 제어 아이콘에 숫자가 표시되는 건 저장소에 변경 사항이 있다는 의미로 지금은 변경 사항이 1개 있음
소스 제어 아이콘을 클릭해 보면 '변경 사항'이라는 영역에 저장소의 변경 내용이 나타남
방금 work1.txt 파일을 만들었기 때문에 [변경 사항] 영역에 work1.txt 파일이 나타나 있음
2. work1.txt 파일 이름 위로 마우스 포인터를 올리면 오른쪽에 아이콘 4개가 표시됨



- ① **파일 열기:** 편집 창에 현재 파일을 열어 줌
지금은 work1.txt 파일이 열려 있는 상태이므로 클릭해도 변화가 없음
- ② **변경 내용 취소:** 파일의 변경 사항을 취소
- ③ **변경 내용 스테이징:** 현재 파일을 스테이지에 올림
- ④ **파일의 상태:** 처음에 U라고 표시된 것은 Untracked 파일이라는 의미
아직 한번도 커밋하지 않은 파일이라는 뜻
한번 커밋한 후에 변경 사항이 생기면 M으로 표시됨

스테이징하기

1. [변경 사항]에 있는 파일을 스테이지에 올려놓으려면, 파일 이름 위로 마우스 포인터를 올렸을 때 오른쪽에 나타나는 아이콘 4개 중에서 **+**를 클릭
2. [변경 사항]에 있던 파일이 [스테이징된 변경 사항]으로 스테이지로 옮겨졌음
그리고 스테이징된 파일 오른쪽에 'A'라고 표시되는데 '인덱스 추가됨(added)'이라는 의미
3. 스테이징 영역에 있는 파일 위로 마우스 포인터를 옮겼을 때 역시 다음과 같이 몇 가지 아이콘이 표시됨



- ① **파일 열기**: 스테이지에 있는 파일을 편집 창에 열어 줌
- ② **변경 내용 스테이징 취소**: 클릭하면 스테이징이 취소되고 변경사항 항목으로 이동
- ③ **인덱스 추가됨**: [A]는 added의 줄임말로, 스테이지에 추가된 상태라는 뜻

커밋하기

1. 스테이지에 있는 파일은 커밋할 수 있음

소스 제어 창 맨 위에 있는 메시지 창에 커밋 메시지를 입력한 후 메시지 창 위에 있는  를 클릭

2. 커밋이 끝나면 소스 제어 아이콘에 있던 숫자가 사라짐

한꺼번에 여러 파일 스테이징하고 커밋하기

1. 이번에는 파일 2개를 한꺼번에 스테이징하고 커밋하기 위해 우선 사이드바에서  (탐색기)를 클릭해 탐색기 창을 열고 [PROJECT] 폴더 이름 위로 마우스 포인터를 올린 후  (새 파일)을 클릭해서 work2.txt 파일을 만들고, 파일 내용에는 '2'라고 입력
다시 한번 work3.txt 파일을 만들고 여기에도 마찬가지로 두 번째 작업이라는 뜻으로 '2'라고 입력
2.  (소스 제어)를 클릭하면 [변경 사항]에 파일 2개가 변경되어 있을 것
 를 클릭해서 파일 2개를 스테이지에 각각 올림
3. 두 파일이 모두 [스테이징된 변경 사항] 항목에 추가된 것을 확인
이제 커밋하기 위해 커밋 메시지 창에 메시지를 입력하고  를 클릭해서 커밋
이처럼 파일을 여러 개 스테이지에 올리고 나서 한꺼번에 커밋할 수 있음

스태이징과 커밋 한번에 처리하기

1. work1.txt 파일을 열거나, 이미 열려 있다면 work1.txt 탭을 클릭해서 편집 창으로 이동
그리고 '1' 아래쪽에 세 번째 작업이라는 뜻으로 '3'을 입력하고, 저장
2. 변경 사항이 있으니 에 1이 표시될 것
을 클릭해서 소스 제어 창으로 이동
work1.txt 파일 오른쪽에 M이라고 표시되어 있는데 M은 수정되었다는(modified) 의미
3. M이라고 표시된 변경 사항은 스테이징하지 않은 상태에서 커밋 메시지를 입력하고 ☒를 클릭하면
스태이징과 커밋을 한꺼번에 진행
4. 이때 스테이징 없이 커밋할 것인지 묻는 창이 나타나는데 [예]를 클릭하면 스테이징과 커밋이 곧바로 끝남

변경 내용 살펴보기

1. work2.txt 파일을 편집 창에 열고 '2' 다음에 '4'를 입력하고 파일을 저장한 후 소스 제어 창으로 이동
2. 변경 내용을 확인하려면 [변경 사항]에 있는 파일 이름을 클릭
work2.txt를 선택하면 편집 창에 새로운 탭이 열리면서 작업 트리에 있는 work2.txt 파일을 보여 줌
왼쪽에는 기존 파일 내용이 나타나고, 오른쪽에는 수정한 파일 내용이 보임
두 번째 줄에 '4'가 추가되었다는 것을 알 수 있음

변경 내용 취소하기

1. 만일 변경 내용을 취소하겠다면 소스 제어 창에서 work2.txt 파일 오른쪽에 있는 아이콘 중에서  를 클릭
이어서 변경 내용을 취소할 것인지 묻는 창이 나타나면 [변경 내용 취소]를 클릭
2. 변경 내용이 취소되어 work2.txt는 '2'만 남아 있고 소스 제어 창에 있던 숫자 '1'도 사라짐
즉, '4'를 추가하기 전의 상태로 되돌아감

터미널 창에서 깃 로그 살펴보기

- [터미널] → [새 터미널]을 선택해 터미널 창을 열어서 'git log'라고 입력하면 지금까지 만든 커밋들이 나타남
- 커밋 중에서 (HEAD -> main)이라고 된 부분이 가장 최신
- 이렇게 필요할 때마다 터미널 창을 열어 앞에서 공부했던 깃 관련 명령들도 함께 사용할 수 있음

깃허브에 원격 저장소 만들기

1. 우선 연습에 사용할 원격 저장소부터 만들어야 함
깃허브에 로그인하면 첫 화면 왼쪽에 저장소 목록이 나타남
목록 위에 있는 [New]를 클릭
2. 여기에서는 'test-3'라는 이름으로 저장소를 만들어 봄
3. 저장소가 만들어지면 접속할 수 있는 저장소 주소가 나타남
주소 오른쪽에서 를 클릭해 HTTPS 접속 주소를 복사

원격 저장소 연결하기

1. 원격 저장소를 연결하기 위해  을 클릭해서 소스 제어 창을 연 뒤  을 클릭해 [원격] → [원격 추가]를 선택
2. 화면 위쪽의 입력 창에 복사한 원격 저장소 주소를 입력하고 {{ Enter }}를 누른 후,
원격 저장소 이름(test-3)도 입력하고 다시 {{ Enter }}를 누름
원격 저장소 이름을 지정하는 것은 매번 주소를 기억해 두었다가 사용하기 어렵기 때문
여기에서 원격 저장소 주소를 한 번만 추가하면 이름을 사용해 저장소를 구별할 수 있도록 한 것
3. 원격 저장소에 연결되면 정기적으로 fetch할 것인지 묻는 알림 창이 나타남
중요한 프로젝트라면 [예]를, 자주 폐치할 필요가 없다면 [아니요]를 선택

원격 저장소로 푸시하기

1. 소스 제어 창을 열고  을 클릭해서 [풀, 푸시] → [다음으로 푸시]를 선택
2. 원격 저장소에 연결되면 지역 저장소(작업 폴더)에 있던 파일들이 깃허브로 바로 푸시됨
여기에서는 파일이 몇 개 안 되어서 푸시는 금방 끝남
푸시하는 동안에는 화면 아래 상태 표시줄에 있는  아이콘이  로 바뀌는데,
이 아이콘이 다시  으로 바뀌면 푸시가 끝났다는 뜻
이제 웹 브라우저에서 깃허브 저장소로 접속해 보면 아무 내용도 없던 화면에 방금 푸시한 파일들이 올라와 있음

새로운 브랜치 만들기

1. 소스 제어 창에서  을 클릭하고 [분기] → [분기 만들기]를 선택한 후 새로운 브랜치 이름을 입력
여기에서는 'newBranch'라고 함
2. newBranch 브랜치가 만들어지면서 그 브랜치로 전환됨
VS Code의 상태 표시줄을 보면 처음에 main이었던 브랜치가 새로 만든 브랜치, 여기에서는 newBranch로 바뀌어 있을 것
즉, 현재 newBranch에 있다는 뜻

새 브랜치에서 커밋하기

1. 사이드바에서  를 클릭해 탐색기 창으로 이동한 후, 현재 브랜치에 work4.txt 파일을 만들고 '4'를 입력
2. 방금 만든 파일을 커밋하기 위해  를 클릭해 소스 제어 창으로 이동
Untracked 파일이므로  를 클릭해서 스테이징
3. 커밋 메시지를 입력한 후  를 클릭해 커밋
지금 만든 커밋은 newBranch의 커밋임

브랜치 사이에 전환하기

1. VS Code의 상태 표시줄을 보면 가장 왼쪽에 브랜치 이름이 나타남
이 부분을 클릭하면 화면 상단에 브랜치 이름이 나타나는데, 그중에서 main을 선택
2. 상태 표시줄을 보면 브랜치가 main으로 바뀐 것을 확인할 수 있음
3. newBranch에서 작업했던 work4.txt는 main 브랜치에서는 볼 수 없게 됨
왜냐하면 work4.txt 파일은 newBranch에서 만들어서 커밋한 파일이기 때문

새 브랜치를 원격 저장소로 푸시하기

1. 소스 제어 창에서  을 클릭하고 [풀, 푸시] → [다음으로 푸시]를 선택한 후, 푸시할 원격 저장소를 선택
여기에서는 test-3.git을 선택하면 됨
따로 브랜치를 선택하거나 입력하지 않고 간단하게 푸시됨
2. 이제 깃허브에서 원격 저장소로 접속하고 살펴봄
저장소의 기본 브랜치는 main 브랜치이기 때문에 처음에는 main 브랜치의 내용만 나타남
그러므로 work3.txt 파일까지만 보일 것임
3. 원격 저장소에서 main 부분을 클릭하면 저장소에 있는 여러 브랜치가 나타남
그중에서 새로 만든 newBranch 브랜치를 선택
4. 원격 저장소에 새로운 브랜치와 브랜치 안의 내용이 제대로 푸시된 것을 확인할 수 있음

브랜치 병합하기

1. 소스 제어 창에서  을 클릭하고 [분기] → [분기 병합]을 선택
2. 병합할 브랜치를 선택, 여기에서는 newBranch를 선택
3. 이제 newBranch가 main 브랜치로 병합되면서, newBranch에서 만들었던 work4.txt를 main 브랜치에서도 볼 수 있음